

WEEK-4 PROJECT REPORT

Business Analytics & DBMS

BA Task: High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch

DBMS Task: Order Management System

Student Name: _____

Class: B.Sc. AI & ML

Submitted to: Ruchi Mam

PART A — BUSINESS ANALYTICS PROJECT

High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch

1. Introduction

Market Basket Analysis is a frequent pattern mining technique used to discover products that are commonly purchased together. This project implements an FP-Growth style mining engine from scratch using a compressed FP-Tree. The tree reduces repeated transaction information and supports efficient discovery of frequent itemsets without generating a large candidate set.

2. Objectives

- Build a frequent pattern mining engine from scratch.
- Construct a compressed FP-Tree using dynamic nodes.
- Use node-link pointers to connect equal items in the tree.
- Generate conditional pattern bases and conditional FP-Trees recursively.
- Extract frequent itemsets and high-confidence association rules.
- Evaluate the mining process using support and confidence measures.

3. Dataset

Transaction	Items
T1	Bread, Milk
T2	Bread, Diaper, Beer, Eggs
T3	Milk, Diaper, Beer, Coke
T4	Bread, Milk, Diaper, Beer
T5	Bread, Milk, Diaper, Coke

4. Important Concepts

Support: $\text{Support}(X) = \text{Number of transactions containing } X / \text{Total number of transactions}$.

Confidence: $\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$.

FP-Tree: A compressed prefix tree that stores frequent transaction items in shared paths.

Node Link: A pointer connecting nodes representing the same item, enabling efficient traversal.

5. Methodology

- Read the transaction database.
- Count the frequency of every item.
- Remove items below the minimum support threshold.
- Sort each transaction by descending frequency.
- Insert sorted transactions into the FP-Tree.
- Create header-table node links for repeated items.
- Select an item and construct its conditional pattern base.
- Build the conditional FP-Tree recursively.
- Generate frequent itemsets.

- Generate association rules and retain rules meeting the minimum confidence.

6. Python Implementation

```
from collections import defaultdict

transactions = [
    ["bread", "milk"],
    ["bread", "diaper", "beer", "eggs"],
    ["milk", "diaper", "beer", "coke"],
    ["bread", "milk", "diaper", "beer"],
    ["bread", "milk", "diaper", "coke"]
]

min_support = 2

freq = defaultdict(int)
for t in transactions:
    for item in t:
        freq[item] += 1

freq = {k:v for k,v in freq.items() if v >= min_support}

class Node:
    def __init__(self, item, count=1, parent=None):
        self.item = item
        self.count = count
        self.parent = parent
        self.children = {}
        self.link = None

def insert_tree(items, root, header):
    node = root
    for item in items:
        if item not in node.children:
            node.children[item] = Node(item, 1, node)
            if header[item]:
                current = header[item]
                while current.link:
                    current = current.link
                current.link = node.children[item]
            else:
                header[item] = node.children[item]
        else:
            node.children[item].count += 1
            node = node.children[item]

root = Node("NULL", 0)
header = {item: None for item in freq}

for t in transactions:
    ordered = sorted(
        [i for i in t if i in freq],
        key=lambda x: freq[x],
        reverse=True
    )
    insert_tree(ordered, root, header)

print("Frequent items:", freq)
print("FP-Tree construction completed.")
```

7. Association Rule Example

If the itemset {Bread, Milk, Diaper} is frequent, a possible rule is **Bread, Milk** → **Diaper**. Its confidence is calculated as $\text{Support}(\text{Bread, Milk, Diaper}) / \text{Support}(\text{Bread, Milk})$. Only rules whose confidence reaches the chosen threshold are reported.

8. Expected Output

- Frequent item counts.
- Compressed FP-Tree successfully constructed.

- Frequent itemsets discovered through recursive conditional mining.
- Association rules displayed with support and confidence.
- High-confidence rules selected for business interpretation.

9. Business Interpretation

The discovered associations can support product bundling, shelf arrangement, cross-selling, recommendation systems, and promotional planning. The compressed FP-Tree approach is useful when transaction datasets become large because common prefixes are shared.

10. Conclusion

The project demonstrates a from-scratch frequent pattern mining engine based on compressed FP-Trees, node links, conditional pattern bases, and recursive mining. Support and confidence provide a quantitative basis for selecting useful association rules.

PART B — DBMS PROJECT

Order Management System

1. Introduction

The Order Management System is a relational database application for recording customer orders and the products included in each order. It stores order dates, quantities, product prices, total amounts, and supports insertion, modification, and customer order-history reporting.

2. Objectives

- Design Orders and Order_Details tables.
- Maintain customer and product order information.
- Store order date, quantity, unit price, and total amount.
- Perform order insertion and modification operations.
- Generate customer order history reports using SQL joins.

3. Database Design

Table	Important Fields	Purpose
Customers	customer_id (PK), customer_name, phone, email	Stores customer details
Products	product_id (PK), product_name, price	Stores product details
Orders	order_id (PK), customer_id (FK), order_date, total_amount	Stores order header
Order_Details	order_detail_id (PK), order_id (FK), product_id (FK), quantity, unit_price	Stores product price in each order

4. SQL Table Creation

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    phone VARCHAR(15),  
    email VARCHAR(100)  
);  
  
CREATE TABLE Products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    price DECIMAL(10,2) NOT NULL  
);  
  
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE NOT NULL,  
    total_amount DECIMAL(10,2),  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)  
);  
  
CREATE TABLE Order_Details (  
    order_detail_id INT PRIMARY KEY,  
    order_id INT,  
    product_id INT,  
    quantity INT NOT NULL,  
    unit_price DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);
```

5. Sample Data

Customer ID	Customer Name	Phone
-------------	---------------	-------

1	Arun	9876543210
2	Kavi	9876501234

```
INSERT INTO Customers VALUES
(1, 'Arun', '9876543210', 'arun@example.com'),
(2, 'Kavi', '9876501234', 'kavi@example.com');
```

```
INSERT INTO Products VALUES
(101, 'Laptop', 55000.00),
(102, 'Keyboard', 1200.00),
(103, 'Notebook', 80.00);
```

```
INSERT INTO Orders VALUES
(1001, 1, '2026-08-15', 56200.00);
```

```
INSERT INTO Order_Details VALUES
(1, 1001, 101, 1, 55000.00),
(2, 1001, 102, 1, 1200.00);
```

6. Order Insertion

A new order is inserted into Orders, followed by its individual products in Order_Details. The total amount can be calculated from quantity × unit price.

7. Order Modification

```
UPDATE Order_Details
SET quantity = 2
WHERE order_detail_id = 2;

UPDATE Orders
SET total_amount = (
    SELECT SUM(quantity * unit_price)
    FROM Order_Details
    WHERE order_id = 1001
)
WHERE order_id = 1001;
```

8. Customer Order History Report

```
SELECT
    c.customer_name,
    o.order_id,
    o.order_date,
    p.product_name,
    od.quantity,
    od.unit_price,
    (od.quantity * od.unit_price) AS item_total,
    o.total_amount
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Order_Details od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
ORDER BY c.customer_name, o.order_date;
```

9. Expected Output

Customer	Order ID	Date	Product	Qty	Item Total
Arun	1001	2026-08-15	Laptop	1	55000.00
Arun	1001	2026-08-15	Keyboard	1	1200.00

10. Advantages

- Maintains organized customer order records.
- Reduces duplicate order information through normalization.
- Uses primary and foreign keys to maintain referential integrity.

- Makes order modification and reporting easier.
- Supports customer-wise order history analysis.

11. Conclusion

The Order Management System provides a structured DBMS solution for managing customer orders and order details. The relational design, SQL operations, and customer history report demonstrate practical database management skills.

Overall Conclusion

The Week-4 projects combine Business Analytics and DBMS skills. The BA project focuses on frequent pattern mining and association-rule discovery, while the DBMS project focuses on relational order processing and reporting. Both projects can be extended with larger datasets, dashboards, and real-time applications.